

OS Orange

Name: Shriya S R

SRN: PES1UG24CS449

Class: 4 H

SS 1A

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

SS 1B

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

SS 2A

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

SS 2B

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a          egriXy.
```

SS 3A

```

shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ nano index.c
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ make
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes add file1.txt file2.txt
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: .gitignore
  untracked: test_objects.c
  untracked: README.md
  untracked: Makefile
  untracked: tree.h
  untracked: pes.c
  untracked: commit.h
  untracked: commit.c
  untracked: test_tree.c
  untracked: pes.h
  untracked: test_sequence.sh
  untracked: tree.c
  untracked: a.txt
  untracked: index.c
  untracked: object.c
  untracked: index.h

```

SS 3B

```

shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776437042 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776437046 6 file2.txt

```

SS 4A

```

shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ nano commit.c
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ make
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ echo "Hello" > hello.txt
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes add hello.txt
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 723c47806ec2... Initial commit
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ echo "World" >> hello.txt
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes add hello.txt
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes commit -m "Second commit"
Committed: ea1f5f4c9f65... Second commit
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ ./pes log
commit ea1f5f4c9f6557c5c453b15730dfa20ba05e36f0f2c568c71667f4ce925f2f48
Author: PES User <pes@localhost>
Date: 1776437979

    Second commit

commit 723c47806ec2141f3406618ff58043ec9597e8a390224931d472902baae60224
Author: PES User <pes@localhost>
Date: 1776437962

    Initial commit

```

SS 4B

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/72/3c47806ec2141f3406618ff58043ec9597e8a390224931d472902baae60224
.pes/objects/ea/1f5f4c9f6557c5c453b15730dfa20ba05e36f0f2c568c71667f4ce925f2f48
.pes/refs/heads/main
```

SS 4C

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ cat .pes/refs/heads/main
ea1f5f4c9f6557c5c453b15730dfa20ba05e36f0f2c568c71667f4ce925f2f48
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final

```
shriya_pes1ug24cs449@shriyapes1ug24cs449:~/PES1UG24CS449-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed: 3ef6e9eea98b... Initial commit

Log after first commit:
commit 3ef6e9eea98b03b5d4170cd527e148cf5b64541929f8e817266a522c43dce079
Author: PES User <pes@localhost>
Date:   1776438489

    Initial commit

--- Second Commit ---
Committed: 4767ef59de43... Update file.txt

--- Third Commit ---
Committed: a6c62eb02d44... Add farewell
```

```

--- Full History ---
commit a6c62eb02d449a2ce8e84e812ec5e69f4bbc0c74434bb74bc801eeec44cb57f2
Author: PES User <pes@localhost>
Date: 1776438489

    Add farewell

commit 4767ef59de43dcacbf4b9e49522fdca7ad8e9fb04f7b32db8d72aeb068a7dc24
Author: PES User <pes@localhost>
Date: 1776438489

    Update file.txt

commit 3ef6e9eea98b03b5d4170cd527e148cf5b64541929f8e817266a522c43dce079
Author: PES User <pes@localhost>
Date: 1776438489

    Initial committial commiq

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
a6c62eb02d449a2ce8e84e812ec5e69f4bbc0c74434bb74bc801eeec44cb57f2

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/3e/f6e9eea98b03b5d4170cd527e148cf5b64541929f8e817266a522c43dce079
.pes/objects/47/67ef59de43dcacbf4b9e49522fdca7ad8e9fb04f7b32db8d72aeb068a7dc24
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/a6/c62eb02d449a2ce8e84e812ec5e69f4bbc0c74434bb74bc801eeec44cb57f2
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0cccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===

```

Phase 5 & 6: Analysis-Only Questions

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

A) To implement checkout, two major systemic areas must be updated:

1. **The .pes/ Directory:** The `.pes/HEAD` file must be updated to point to the new branch (e.g., its contents change from `ref: refs/heads/main` to `ref: refs/heads/feature`). Additionally, the `.pes/index` (staging area) must be completely overwritten to match the flat file list and blob hashes of the target branch's commit tree.
2. **The Working Directory:** The actual files on your hard drive must physically change to mirror the target commit's snapshot.
 - Files that exist in the target commit but not currently must be created.
 - Files that exist currently but not in the target commit must be deleted.
 - Files that exist in both but differ must be overwritten with the blob contents from the target commit.

Why it is complex: The complexity arises from **safety and filesystem manipulation**. You aren't just changing pointers; you are executing bulk I/O operations. You must safely traverse nested directories, create missing folders, and set proper file permissions (`chmod`). Most importantly, you have to ensure you do not accidentally overwrite or delete untracked local files or uncommitted changes without warning the user (which leads into Q5.2).

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A) To detect a dirty working directory conflict, you must cross-reference three states: the Working Directory, the Staging Area (.pes/index), and the Target Branch Tree (.pes/objects/).

1. **Identify Modified Tracked Files:** First, compare the working directory files to the entries in the index. If a tracked file's size or modified timestamp (mtime) differs from what is in the index, or if hashing its current contents yields a different hash than the index, the file is "dirty" (modified).
2. **Compare Current Index vs. Target Tree:** Next, recursively walk the tree object of the *target* branch to find the expected blob hash for that specific file path.
3. **Evaluate the Conflict:**
 - If the file is dirty locally, but the target branch's tree has the *exact same blob hash* for that file as your current commit's tree, there is no conflict. You can safely switch branches and leave the user's local modification untouched.
 - **The Conflict:** If the file is dirty locally, AND the target branch expects a *different* blob hash for that file, you must abort. If you checked out, you would overwrite the user's unsaved local work with the target branch's version, destroying their data.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

A) **What happens:** If you create a commit in a Detached HEAD state, the commit process works exactly as normal: a new tree is built, a new commit object is written to the object store, and its parent points to the previous commit. Finally, .pes/HEAD is updated with the new commit's hash. However, because HEAD is not pointing to a branch file (like refs/heads/main), **no branch reference is updated to track this new commit.**

How to recover: If the user subsequently checks out an actual branch (e.g., pes checkout main), HEAD is overwritten, and those detached commits become "orphaned" because no branch reference points to them. To recover them, the user simply needs to find the hash of that orphaned commit (real Git provides git reflog for this, but the user could also just scroll up in their terminal history to find the hash they committed). Once they have the hash, they can recover it by manually creating a new branch file in refs/heads/ containing that hash (e.g., pes branch recovery-branch <hash>).

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

A) **The Algorithm (Mark-and-Sweep):**

1. **Mark Phase:** Start from all known entry points: read every file in `.pes/refs/heads/` and `.pes/HEAD` to get active commit hashes. For each commit, read its parent pointer to walk backwards through history. For every commit visited, walk its tree recursively. Add the hash of every commit, tree, and blob encountered into a "reachable" data structure.
2. **Sweep Phase:** Iterate through every physical file in the `.pes/objects/` directory. If a file's hash name is *not* present in the reachable data structure, it is orphaned. Delete it.

Data Structure: A Hash Set (or a Bloom filter paired with a Set for memory optimization) is the best choice. It allows for $O(1)$ constant-time lookups to instantly check if a hash has already been marked, preventing infinite loops and duplicate processing.

Estimation:

For 100,000 commits, assuming average repository churn where a commit touches a few files and generates a new root tree, you have:

- 100,000 commit objects.
- 100,000 root tree objects.
- Thousands of shared subtrees and blobs.

Because Git uses content-addressable storage (deduplication), if you prune your traversal—meaning you stop walking a tree if its hash is already in your Hash Set—you only visit each *unique* object once. A repository with 100,000 commits could roughly contain anywhere from **500,000 to 1.5 million unique objects** to visit and mark.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

A) **Why it is dangerous:** Garbage Collection operates under the assumption that if an object isn't linked to a reference, it is trash. However, during a commit operation, objects are actively being written to disk *before* they are officially linked to a commit or branch.

The Race Condition:

1. **Action:** The user runs `pes add main.c`.
2. **Result:** A new blob Hash-A is written to `.pes/objects/`. It is currently unreferenced by any commit.
3. **Action:** A background Garbage Collection process starts. It builds its "reachable" list.
4. **Action:** The user types `pes commit`. The commit logic starts building trees that will point to Hash-A.
5. **Result:** GC enters its Sweep phase. It sees Hash-A in the object store but not in its reachable list. It deletes Hash-A.
6. **Crash:** The commit operation finishes, pointing to a blob that no longer exists. The repository is now corrupted.

How real Git avoids this: Git avoids this race condition using a Timestamp Grace Period. When `git gc` runs, it checks the modification time (`mtime`) of unreachable objects on the filesystem. By default, Git refuses to delete any loose object that is less than 2 weeks old.

Even if Hash-A is currently unreachable, its file timestamp proves it was just created seconds ago, so GC safely ignores it, giving the user plenty of time to finish their commit.